

Pandas for Machine Learning

Практический конспект - DataFrame - Cleaning - GroupBy - Features - Leakage

Главная идея: Pandas превращает сырую таблицу в понятные и проверенные признаки, но все обучаемые преобразования должны остаться внутри train-only ML workflow.

Содержание

1. Series и DataFrame	1
2. CSV и первичная инспекция	2
3. Выбор данных: [], loc и iloc	2
4. Фильтрация строк	2
5. Пропуски NaN	2
6. Дубликаты	3
7. Типы данных	3
8. Сортировка и value_counts	3
9. groupby и agg	3
10. apply: когда он нужен	3
11. merge и ключи	4
12. Категориальные данные	4
13. Feature engineering	4
14. Data leakage	4
15. Практический Pandas workflow	5
16. Финальная шпаргалка	5
17. Что должно быть в голове	5

1. Series и DataFrame

Series - одномерный массив с индексом. **DataFrame** - таблица: строки являются объектами, столбцы - признаками. Индекс помогает адресовать строки, но не обязан совпадать с их позицией.

```
import pandas as pd

s = pd.Series([18, 21, 20], name='age')
df = pd.DataFrame({'age': [18, 21], 'city': ['Moscow', 'Perm']})

df.shape
df.index
df.columns
```

ML: Обычно одна строка DataFrame - один sample, один столбец - один feature.

2. CSV и первичная инспекция

```
df = pd.read_csv('students.csv')

df.head()
df.tail()
df.sample(5, random_state=42)
df.shape
df.info()
df.describe(include='all')
```

Проверка	Что узнаём
shape	Сколько строк и столбцов
info()	Типы, non-null, память
describe()	Распределения и частоты
head()/sample()	Как данные выглядят глазами

Правило: До любых преобразований проверь размер, названия столбцов, типы, пропуски и несколько реальных строк.

3. Выбор данных: [], loc и iloc

```
df['age']          # Series
df[['age', 'score']] # DataFrame

df.loc[10, 'age']  # labels
df.loc[df['age'] >= 18, ['age', 'city']]
df.iloc[0:5, 0:3]  # positions
```

Критическая разница: **loc** работает по меткам и включает обе границы label-среза; **iloc** работает по позициям и не включает правую границу.

4. Фильтрация строк

```
mask = (df['age'] >= 18) & (df['score'] >= 70)
adults = df.loc[mask]

df[df['city'].isin(['Moscow', 'Perm'])]
df[df['name'].str.contains('ann', case=False, na=False)]
df[df['score'].between(60, 80)]
```

Важно: Каждое сравнение заключай в скобки. Используй **&**, **|**, **~**, а не Python-операторы **and/or/not**.

5. Пропуски NaN

```
df.isna().sum().sort_values(ascending=False)

df['age'] = df['age'].fillna(df['age'].median())
df['city'] = df['city'].fillna('Unknown')
df = df.dropna(subset=['target'])
```

Числовой признак часто заполняют медианой, категориальный - модой или отдельной категорией. Удаление строк оправдано только после оценки масштаба потерь.

Leakage: Значение для заполнения обучается только на train. В финальном ML workflow это делает **SimpleImputer** внутри Pipeline.

6. Дубликаты

```
df.duplicated().sum()
df[df.duplicated(keep=False)]

df = df.drop_duplicates()
df = df.drop_duplicates(subset=['student_id'], keep='last')
```

Перед удалением: Определи, что считается дублем: полная строка, идентификатор или комбинация бизнес-полей.

7. Типы данных

```
df.dtypes

df['age'] = pd.to_numeric(df['age'], errors='coerce')
df['date'] = pd.to_datetime(df['date'], errors='coerce')
df['city'] = df['city'].astype('category')
df['passed'] = df['passed'].astype('boolean')
```

Ошибка: Столбец object может скрывать числа, даты и смешанные значения. Неверный dtype ломает сортировку, агрегации и признаки.

8. Сортировка и value_counts

```
df.sort_values('score', ascending=False)
df.sort_values(['city', 'score'], ascending=[True, False])

df['city'].value_counts()
df['city'].value_counts(normalize=True, dropna=False)
```

value_counts(normalize=True) возвращает доли. Это быстрый способ увидеть дисбаланс категорий и target.

9. groupby и agg

```
summary = (
    df.groupby('city', dropna=False)
      .agg(
          students=('student_id', 'nunique'),
          mean_score=('score', 'mean'),
          pass_rate=('passed', 'mean'),
      )
      .sort_values('mean_score', ascending=False)
)
```

Модель мышления: split -> apply -> combine: разделить строки по группам, посчитать функцию, собрать результат.

10. apply: когда он нужен

```
def age_group(age):
    if pd.isna(age): return 'unknown'
    if age < 18: return 'minor'
    return 'adult'

df['age_group'] = df['age'].apply(age_group)
```

Сначала ищи векторное решение: **map**, **replace**, **where**, **cut**, строковые и datetime-методы. **apply** удобен, но часто медленнее и сложнее для проверки.

```
df['city_code'] = df['city'].map({'Moscow': 1, 'Perm': 2})
df['age_group'] = pd.cut(df['age'], bins=[0, 17, 25, 100])
```

11. merge и ключи

```
result = students.merge(
    exams,
    on='student_id',
    how='left',
    validate='one_to_many',
    indicator=True,
)
```

how	Что остаётся
inner	Совпавшие ключи
left	Все строки слева
right	Все строки справа
outer	Все ключи обеих таблиц

Проверка: Сверяй число строк до/после, уникальность ключей и `_merge`. Неожиданное размножение строк обычно означает many-to-many.

12. Категориальные данные

```
df['city'] = df['city'].astype('category')
df['city'].cat.categories

df = pd.get_dummies(df, columns=['city'], dtype=int)
```

Для исследования **category** экономит память и фиксирует набор значений. Для модели категории кодирует **OneHotEncoder** внутри sklearn Pipeline, чтобы train и новые данные преобразовывались одинаково.

13. Feature engineering

```
df['score_per_hour'] = df['score'] / df['study_hours'].replace(0, pd.NA)
df['age_squared'] = df['age'] ** 2
df['exam_month'] = df['exam_date'].dt.month
df['is_weekend'] = df['exam_date'].dt.dayofweek.ge(5).astype(int)
```

Хороший признак вычисляется одинаково для train и будущего объекта, доступен в момент предсказания и имеет понятный смысл.

Leakage: Нельзя создавать feature из target, будущего результата или статистики, рассчитанной на всём датасете.

14. Data leakage

Опасно	Безопасно
Заполнить NaN до split	Imputer.fit только на train
Масштабировать весь df	Scaler внутри Pipeline
Средний target по всей категории	Расчёт только внутри train/CV
Признак после события	Только доступное на момент прогноза

Главный тест: Спроси: мог бы я честно вычислить этот столбец для нового объекта в момент предсказания? Если нет - это leakage.

15. Практический Pandas workflow



```

df = pd.read_csv('students.csv')
print(df.shape)
print(df.info())

df = df.drop_duplicates(subset='student_id')
df['exam_date'] = pd.to_datetime(df['exam_date'], errors='coerce')
df['study_intensity'] = df['score'] / df['study_hours'].replace(0, pd.NA)

X = df.drop(columns='passed')
y = df['passed']
  
```

16. Финальная шпаргалка

```

# inspect
df.shape; df.head(); df.info(); df.describe(include='all')

# select / filter
df.loc[(df['age'] >= 18) & df['city'].notna(), ['age', 'city']]

# quality
df.isna().sum(); df.duplicated().sum(); df.dtypes

# summarize
df['city'].value_counts(normalize=True, dropna=False)
df.groupby('city').agg(mean_score=('score', 'mean'))

# combine
left.merge(right, on='id', how='left', validate='one_to_one')
  
```

17. Что должно быть в голове

Pandas: строки = samples; столбцы = features; index = labels; loc = labels; iloc = positions

Порядок: inspect -> validate -> clean -> explore -> engineer features -> X/y -> sklearn Pipeline

Контроль: shape, NaN, duplicates, dtypes и ключи merge проверяются до и после каждого существенного изменения.